GBD · Teoría 3 · SQL: consultas

Teoría del RA4 (consultar la información): cómo se le pregunta a una base de datos con `SELECT`, de la consulta más simple a las combinaciones de tablas y las subconsultas. Resume la unidad 4 del IOC, *Llenguatge SQL. Consultes*, con MariaDB en lugar de MySQL y todos los ejemplos sobre la base de la liga.

Apartado	Qué	IOC, unidad 4
1	SQL: origen, tipos de sentencias, tipos de datos, cómo se escribe	1.1–1.3 (págs. 223–235)
2	`SELECT`, `FROM`, alias, `ORDER BY`, `LIMIT`, `WHERE`, `DISTINCT`	1.4, 2.2, 2.3 (págs. 235–252, 265–267)
3	Funciones: números, texto, fechas, control de flujo	2.1 (págs. 253–265)
4	Funciones de grupo, `GROUP BY`, `HAVING`	2.4 (págs. 267–270)
5	`UNION`, `INTERSECT`, `EXCEPT`	2.5 (págs. 270–272)
6	Combinaciones: producto cartesiano, `JOIN`, `LEFT JOIN`, la misma tabla dos veces	2.6 (págs. 272–280)
7	Subconsultas: escalares, `IN`, `ANY`/`ALL`, correlacionadas, `EXISTS`, en el `FROM`	2.7 (págs. 280–285)
Anexo	Trabajar con MariaDB y DBeaver	3 (págs. 285–302)

Al final, un glosario y una autoevaluación con respuestas.

Criterios del RA4 que cubre: a (herramientas y sentencias) en el apartado 1 y el anexo; b (consultas de una tabla) en el 2 y el 3; c (valores de resumen) en el 4; d (composiciones internas) en el 6; e (composiciones externas) en el 6.5; f (subconsultas) en el 7, y g (valorar alternativas) en el 7.7. La f y la g se trabajan en el Reto 1.

Para ampliar: el material del IOC (en catalán, 2011), que tiene el profesor. Usa MySQL 5 y dos bases de ejemplo propias (empresa y sanidad). Es de 2011 y hay cosas que han cambiado: están señaladas con ⚠️ y resumidas en lo que ha cambiado desde el IOC.

La base de ejemplo: la liga

Todos los ejemplos usan la base `liga` (`sql-1/liga.sql`): una liga de fútbol sala con 6 equipos, 48 jugadores, 15 partidos y 68 goles. Para probarlos, se ejecuta `liga.sql` en DBeaver (ver el anexo).

```
EQUIPO(id_equipo PK, nombre UK, ciudad, pabellon?)
JUGADOR(id_jugador PK, nombre, posicion, dorsal, fecha_nacimiento, id_equipo FK → EQUIPO)
        UK(id_equipo, dorsal)
PARTIDO(id_partido PK, jornada, fecha, id_local FK → EQUIPO, id_visitante FK → EQUIPO,
        goles_local, goles_visitante)
GOL(id_gol PK, id_partido FK → PARTIDO, id_jugador FK → JUGADOR, minuto)
```

id_equipo	nombre	ciudad
1	CD Moncayo	Tarazona
2	Atlético Ebro	Zaragoza
3	Huesca Sala	Huesca
4	Teruel Mudéjar FS	Teruel
5	Delicias FS	Zaragoza
6	Cinco Villas FS	Ejea de los Caballeros

Posiciones: `portero`, `cierre`, `ala`, `pívot`. Los partidos duran 40 minutos y se juegan en cinco jornadas, un sábado por semana, del 19 de septiembre al 17 de octubre de 2026.

1 · SQL y sus sentencias

IOC, unidad 4, apartados 1.1 a 1.3 (págs. 223–235).

1.1 De dónde viene

Año	Qué
1970	Edgar F. Codd, en IBM, publica el modelo relacional
1974–1979	IBM construye System R, un gestor de prueba, con un lenguaje llamado SEQUEL (*Structured English Query Language*). Se acorta a **SQL** porque «Sequel» era una marca registrada
1979	Oracle saca el primer gestor relacional comercial con SQL
1986–1987	ANSI e ISO publican el primer estándar: **SQL-86**
1992	**SQL-92**: la gran revisión. Trae el `JOIN` que se usa hoy
1999 en adelante	SQL:1999, 2003, 2011, 2016, 2023: consultas recursivas, disparadores, XML, JSON…

Hay un estándar, pero cada gestor añade y quita cosas: lo que sirve en MariaDB no siempre sirve en PostgreSQL u Oracle. Aquí se usa el SQL de **MariaDB**, que es casi igual al de MySQL.

Se pronuncia letra a letra, «ese, cu, ele». «Sícuel» se oye mucho, pero viene del SEQUEL original.

1.2 Tipos de sentencias

SQL no es solo para preguntar. Sus sentencias se agrupan según lo que hacen:

Grupo	Para qué	Sentencias	Ejemplo
DDL (*Data Definition Language*), definición	Crear, cambiar y borrar la estructura: bases, tablas, claves	`CREATE`, `ALTER`, `DROP`	`CREATE TABLE equipo (…);`
DML (*Data Manipulation Language*), manipulación	Añadir, cambiar y borrar filas	`INSERT`, `UPDATE`, `DELETE`	`DELETE FROM gol WHERE id_gol = 68;`
DQL (*Data Query Language*), consulta	Leer datos	`SELECT`	`SELECT * FROM equipo;`

Grupo	Para qué	Sentencias	Ejemplo
DCL (*Data Control Language*), control	Usuarios y permisos	`GRANT`, `REVOKE`	`GRANT SELECT ON liga.* TO 'ana';`
TCL (*Transaction Control Language*), transacciones	Confirmar o deshacer un grupo de cambios	`COMMIT`, `ROLLBACK`	`ROLLBACK;`

La clasificación cambia de un libro a otro: muchos meten `SELECT` dentro del DML, y el IOC no menciona el TCL. Da igual el nombre del grupo: lo importante es qué hace cada sentencia. Este tema es **solo `SELECT`**. El DDL está en la teoría 4 (diseño físico); el DML y las transacciones, en la teoría 5.

Además de escribirse sueltas, como aquí, las sentencias SQL se pueden meter dentro de un programa (Java, Python, PHP) o dentro de la propia base, en procedimientos con un lenguaje del gestor: PL/SQL en Oracle, Transact-SQL en SQL Server.

1.3 Tipos de datos

Cada columna tiene un tipo, que decide qué valores admite y qué se puede hacer con ellos. Lo básico está en el anexo de la teoría 1; aquí, los de MariaDB:

Familia	Tipo	Qué guarda	Cuándo
Texto	`VARCHAR(n)`	Texto de hasta *n* caracteres. Guarda lo justo	**El habitual** para texto: nombres, códigos
	`CHAR(n)`	Texto de *n* caracteres exactos; rellena con espacios	Códigos de longitud fija: `CHAR(9)` para un DNI
	`TEXT`	Texto largo, sin límite práctico	Observaciones, descripciones
	`ENUM('a','b',…)`	Uno de una lista cerrada	Pocos valores que no cambian: `ENUM('M','T','N')`
	`SET('a','b',…)`	Varios de una lista cerrada	Raro; mejor una tabla aparte
Entero	`INT`	Enteros de unos ±2.100 millones	**El habitual**: identificadores, cantidades
	`TINYINT`, `SMALLINT`, `BIGINT`	Más pequeños o más grandes	Si importa el espacio, o si no cabe en `INT`
Decimal	`DECIMAL(t,d)`	Exacto: *t* cifras en total, *d* decimales	**Dinero** y todo lo que no admita redondeos: `DECIMAL(8,2)` llega a 999999,99
	`FLOAT`, `DOUBLE`	Aproximado	Medidas científicas
Lógico	`BOOLEAN`	Sí o no. En MariaDB es un `TINYINT(1)`: `0` es falso, cualquier otro, verdadero	Marcas: `prestable`
Fecha y hora	`DATE`	`2026-09-19`	Fechas
	`DATETIME`	`2026-09-19 18:30:00`	Fecha con hora
	`TIMESTAMP`	Como `DATETIME`, y se rellena solo con el momento actual	Cuándo se creó o cambió una fila
	`TIME`	`18:30:00`, o una duración	Horas

Familia	Tipo	Qué guarda	Cuándo
	`YEAR`	`2026`	Solo el año

Modificadores de los números:

Modificador	Qué hace	Ojo
`UNSIGNED`	Sin negativos (y llega al doble de alto)	**Solo en números.** `VARCHAR(50) UNSIGNED` da error al crear la tabla
`AUTO_INCREMENT`	El gestor numera solo: 1, 2, 3…	Uno por tabla, y en la clave primaria
`ZEROFILL`	Rellena con ceros a la izquierda al mostrar	Desaconsejado en versiones nuevas

Dos reglas: las fechas siempre en formato año-mes-día y con el año de cuatro cifras (`2026-09-19`); con dos cifras, `69` es 2069 y `70` es 1970.

1.4 Cómo se escribe

```
-- Comentario de una línea
/* Comentario
   de varias líneas */
SELECT nombre, ciudad      -- las palabras clave, en mayúsculas por costumbre
FROM equipo
WHERE ciudad = 'Zaragoza'; -- el texto, entre comillas simples; al final, punto y coma
```

Regla	Detalle
`;` al final	Separa una sentencia de la siguiente. En DBeaver, sin `;` dos consultas seguidas se leen como una
Mayúsculas	A SQL le dan igual en las palabras clave: `select` = `SELECT`. En los nombres de tabla, depende del sistema: en Linux, `Equipo` y `equipo` pueden ser tablas distintas. Mejor todo en minúsculas
Texto y fechas	Entre comillas simples: `'portero'`, `'2026-09-19'`. Los números, sin comillas
Nombres raros	Un nombre con espacios, tildes o que sea palabra reservada, entre comillas invertidas: `` `año` ``. Mejor no tenerlos
Saltos de línea	Dan igual. Una cláusula por línea se lee mejor

En la sintaxis de este documento, lo que va entre corchetes `[ ]` es opcional y `|` separa alternativas.

2 · Consultas simples

IOC, unidad 4, apartados 1.4, 2.2 y 2.3 (págs. 235–252 y 265–267).

Todas las consultas se hacen con una sola sentencia, `SELECT`, que tiene varias partes llamadas **cláusulas**. Se le dice **qué** se quiere, no **cómo** buscarlo: eso lo decide el gestor.

2.1 `SELECT` y `FROM`

```
SELECT expresión [AS alias], expresión [AS alias], …
FROM tabla [alias], …;
```

`SELECT` dice qué columnas salen; `FROM`, de qué tabla.

```
SELECT * FROM equipo;                                -- todas las columnas: 6 filas
SELECT nombre, ciudad FROM equipo;                   -- solo dos
SELECT id_partido, goles_local + goles_visitante AS total
FROM partido;                                        -- una columna calculada, con nombre
```

Elemento	Qué	Ejemplo
`*`	Todas las columnas, en el orden de la tabla	`SELECT * FROM gol;`
Expresión	Una cuenta con columnas, números y funciones. La columna no existe en la tabla: se calcula al consultar	`goles_local - goles_visitante`
Alias de columna	`AS nombre` cambia el título de la columna. El `AS` se puede omitir. Con espacios, entre comillas	`AS \`goles totales\``
Alias de tabla	Un nombre corto para la tabla, después de ella en el `FROM`	`FROM jugador j` y luego `j.nombre`
`base.tabla`	Una tabla de otra base de datos, si hay permiso	`SELECT * FROM liga.equipo;`

Sin `FROM`, `SELECT` sirve de calculadora: `SELECT 4 * 3 - 8 / 2;` da `8.0000`, y `SELECT CURDATE();`, la fecha de hoy. Oracle obliga a escribir `FROM dual`; MariaDB lo admite, pero no hace falta.

Para ver las columnas de una tabla sin leerla entera: `DESCRIBE equipo;` (o `DESC equipo;`). No es SQL estándar: es una orden de MariaDB y MySQL.

2.2 `ORDER BY` y `LIMIT`

El gestor no garantiza ningún orden si no se le pide. `ORDER BY` va después del `WHERE`.

```
SELECT nombre, dorsal FROM jugador
ORDER BY dorsal DESC, nombre;    -- dorsal de mayor a menor; a igual dorsal, por nombre

SELECT nombre, fecha_nacimiento FROM jugador
ORDER BY fecha_nacimiento
LIMIT 3;                         -- los tres más veteranos: Daniel Gil, Hamza Serrano, Youssef Sánchez
```

Detalle	Qué
`ASC`, `DESC`	De menor a mayor (por defecto) o al revés. Va detrás de **cada** columna: `ORDER BY a DESC, b` ordena `a` al revés y `b` normal
Varias columnas	La segunda solo desempata la primera
Por alias o posición	`ORDER BY total DESC` u `ORDER BY 2 DESC` (la segunda columna del `SELECT`)

Detalle	Qué
Por algo que no sale	Se puede ordenar por una columna que no está en el `SELECT`
Texto	En orden alfabético según la **colación** de la base. La liga usa `utf8mb4_spanish_ci`: la ñ va tras la n y las tildes no cuentan
Fechas	De la más antigua a la más reciente. La fecha de nacimiento más baja es la persona **más vieja**
`NULL`	Va **primero** en `ASC` y último en `DESC` ⚠️
`LIMIT n`	Solo las *n* primeras filas. Va siempre al final. No es estándar: SQL Server usa `TOP` y el estándar, `FETCH FIRST n ROWS ONLY`
`LIMIT n OFFSET m`	Se salta *m* filas y da las *n* siguientes. Sirve para paginar

Empates y `LIMIT`. Los tres partidos con más goles son el 6 (9), el 15 (8) y… el 5 o el 12, que tienen 6 cada uno. `LIMIT 3` corta por donde le toca y no avisa. Para que salga siempre lo mismo, un segundo criterio: `ORDER BY total DESC, id_partido`.

2.3 `WHERE`

`WHERE` se queda con las filas que cumplen una condición. Va después del `FROM`.

```
SELECT nombre, dorsal FROM jugador WHERE id_equipo = 4;          -- 8 filas
SELECT * FROM partido WHERE goles_local = goles_visitante;        -- 4 empates
```

Operadores:

Tipo	Operadores	Ejemplo
Aritméticos	`+ - * /`, `%` o `MOD` (resto)	`goles_local + goles_visitante > 5`
Comparación	`=`, `<>` o `!=`, `<`, `>`, `<=`, `>=`	`dorsal > 15`
Lógicos	`AND`, `OR`, `NOT`	`posicion = 'ala' AND id_equipo = 1`
Rango	`[NOT] BETWEEN a AND b`, con los dos extremos incluidos	`minuto BETWEEN 35 AND 40` → 8 goles
Lista	`[NOT] IN (v1, v2, …)`, un `OR` abreviado	`posicion IN ('cierre', 'pívot')`
Patrón	`[NOT] LIKE`, con `%` (cualquier cosa, también nada) y `_` (un carácter exactamente)	`nombre LIKE '%Sánchez'`, `nombre LIKE '_a%'` (segunda letra `a`)
Nulo	`IS [NOT] NULL`	`pabellon IS NULL`
Con subconsulta	`ANY`, `ALL`, `EXISTS`	Ver el apartado 7

`AND` va antes que `OR`, como la multiplicación antes que la suma. «Jugadores de los equipos 1 y 2 que sean alas o porteros»:

```
-- Mal: 18 filas. Se lee «(equipos 1 y 2 y ala) o portero», y salen los 12 porteros de la liga
SELECT nombre FROM jugador
WHERE id_equipo IN (1, 2) AND posicion = 'ala' OR posicion = 'portero';

-- Bien: 10 filas
SELECT nombre FROM jugador
WHERE id_equipo IN (1, 2) AND (posicion = 'ala' OR posicion = 'portero');
```

Ante la duda, paréntesis. El orden completo: `NOT`, luego `AND`, luego `OR`.

La colación también filtra. Con `utf8mb4_spanish_ci`, `LIKE 'A%'` encuentra a Adrián, Aitor y Alejandro, pero también a **Á**ngel y **Á**lvaro: las tildes y las mayúsculas no cuentan. `ci` es *case insensitive*.

`NULL` no es un valor: es «no se sabe». Cualquier comparación con `NULL` da `NULL`, ni verdadero ni falso, y el `WHERE` solo se queda con lo verdadero:

Expresión	Da
`NULL = NULL`	`NULL`
`NULL IS NULL`	`1` (verdadero)
`5 + NULL`	`NULL`
`pabellon = NULL`	Nunca encuentra nada, haya nulos o no
`pabellon IS NULL`	La forma correcta

Fechas. Se comparan como texto entre comillas, en formato año-mes-día: `WHERE fecha BETWEEN '2026-09-26' AND '2026-10-03'` da 6 partidos. ⚠️ El IOC dice que restar dos fechas con `-` da los días entre ellas. En MariaDB no: `DATE '2026-10-17' - DATE '2026-09-19'` da **98**, que es `20261017 − 20260919`. Los días se calculan con `DATEDIFF` (28): ver el apartado 3.3.

2.4 `DISTINCT`

`SELECT DISTINCT` quita las filas repetidas del resultado.

```
SELECT ciudad FROM equipo;            -- 6 filas: Zaragoza sale dos veces
SELECT DISTINCT ciudad FROM equipo;   -- 5 filas
SELECT DISTINCT jornada, fecha FROM partido;   -- 5: una fecha por jornada
```

Con varias columnas, quita las filas en las que **todas** coinciden. Cuesta trabajo al gestor (tiene que ordenar para encontrar repetidos): se usa cuando puede haber repetidos y no interesan, no por si acaso.

2.5 En qué orden se ejecuta

Se escribe en un orden y el gestor lo resuelve en otro. Explica errores que, si no, parecen caprichosos:

Se escribe	Se ejecuta	Qué hace
1 `SELECT`	5	Calcula las columnas y los alias
2 `FROM`	**1**	Junta las tablas
3 `WHERE`	2	Filtra filas

Se escribe	Se ejecuta	Qué hace
4 `GROUP BY`	3	Hace grupos
5 `HAVING`	4	Filtra grupos
6 `ORDER BY`	6	Ordena
7 `LIMIT`	7	Corta

Por eso esto da error:

```
SELECT id_partido, goles_local + goles_visitante AS total
FROM partido
WHERE total > 5;
-- ERROR 1054: Unknown column 'total' in 'WHERE'
```

El `WHERE` se ejecuta antes que el `SELECT`: el alias todavía no existe. Se repite la expresión (`WHERE goles_local + goles_visitante > 5`). En el `ORDER BY`, que va después, el alias sí vale.

3 · Funciones

IOC, unidad 4, apartado 2.1 (págs. 253–265).

Una función recibe valores y devuelve otro. Se usa en el `SELECT`, en el `WHERE` y en el `ORDER BY`. Las de este apartado trabajan **fila a fila**: dan un resultado por fila. Las que resumen varias filas en una (`COUNT`, `SUM`…) son las del apartado 4.

Cada gestor tiene las suyas. Estas son las de MariaDB que más se usan; la lista completa está en la documentación de MariaDB y en el IOC.

3.1 Números

Función	Qué	Ejemplo	Da
`ROUND(x, d)`	Redondea a *d* decimales	`ROUND(4.5333, 1)`	`4.5`
`TRUNCATE(x, d)`	Corta a *d* decimales, sin redondear	`TRUNCATE(4.5999, 1)`	`4.5`
`CEIL(x)`, `FLOOR(x)`	Entero de arriba, entero de abajo	`CEIL(4.2)`, `FLOOR(4.8)`	`5`, `4`
`ABS(x)`	Valor absoluto	`ABS(goles_local - goles_visitante)`	La diferencia sin signo
`MOD(x, y)`, `x % y`	Resto de la división	`MOD(17, 5)`	`2`
`POW(x, y)`, `SQRT(x)`	Potencia, raíz	`POW(2, 10)`	`1024`
`RAND()`	Número al azar entre 0 y 1	`ORDER BY RAND() LIMIT 1`	Un jugador al azar

3.2 Texto

Función	Qué	Ejemplo	Da
`CONCAT(a, b, …)`	Une textos	`CONCAT(nombre, ' (', dorsal, ')')`	`Hamza Serrano (1)`
`CONCAT_WS(sep, a, b, …)`	Une con un separador	`CONCAT_WS(' · ', nombre, ciudad)`	`CD Moncayo · Tarazona`
`UPPER(s)`, `LOWER(s)`	Mayúsculas, minúsculas	`UPPER(nombre)`	`CD MONCAYO`
`CHAR_LENGTH(s)`	Cuántos **caracteres**	`CHAR_LENGTH('Teruel Mudéjar FS')`	`17`
`LENGTH(s)`	Cuántos **bytes**	`LENGTH('Teruel Mudéjar FS')`	`18`: la é ocupa dos en UTF-8
`SUBSTRING(s, desde, cuántos)`	Un trozo. Se cuenta desde 1	`SUBSTRING('Teruel Mudéjar FS', 8, 7)`	`Mudéjar`
`LEFT(s, n)`, `RIGHT(s, n)`	Los *n* primeros, los *n* últimos	`LEFT('Huesca Sala', 6)`	`Huesca`
`SUBSTRING_INDEX(s, sep, n)`	Hasta el *n*-ésimo separador (con *n* negativo, desde el final)	`SUBSTRING_INDEX('Hamza Serrano', ' ', -1)`	`Serrano`
`LOCATE(buscado, s)`	Posición donde aparece; 0 si no está	`LOCATE('FS', 'Cinco Villas FS')`	`14`
`REPLACE(s, viejo, nuevo)`	Sustituye	`REPLACE('Delicias FS', 'FS', 'Fútbol Sala')`	`Delicias Fútbol Sala`
`TRIM(s)`	Quita espacios de los extremos (`LTRIM`, `RTRIM`: solo de un lado)	`TRIM(' x ')`	`x`
`LPAD(s, n, relleno)`	Rellena por la izquierda hasta *n*	`LPAD(7, 3, '0')`	`007`

Para contar letras, `CHAR_LENGTH`. `LENGTH` solo coincide si no hay tildes ni eñes.

3.3 Fechas

Función	Qué	Ejemplo	Da
`CURDATE()`, `NOW()`	Hoy; ahora (con hora)	`CURDATE()`	`2026-10-30`
`YEAR(f)`, `MONTH(f)`, `DAY(f)`	Una parte de la fecha	`MONTH('2026-09-19')`	`9`
`EXTRACT(parte FROM f)`	Lo mismo, en SQL estándar	`EXTRACT(MONTH FROM fecha)`	`9`
`DAYNAME(f)`, `MONTHNAME(f)`	Nombre del día o del mes	`DAYNAME('2026-09-19')`	`Saturday`
`DAYOFWEEK(f)`	Día de la semana, 1 = domingo	`DAYOFWEEK('2026-09-19')`	`7`
`DATEDIFF(f1, f2)`	Días de *f2* a *f1*	`DATEDIFF('2026-10-17', '2026-09-19')`	`28`
`TIMESTAMPDIFF(unidad, f1, f2)`	Diferencia en años, meses, días…	`TIMESTAMPDIFF(YEAR, fecha_nacimiento, CURDATE())`	La edad
`DATE_ADD(f, INTERVAL n unidad)`	Suma (`DATE_SUB` resta)	`DATE_ADD('2026-10-17', INTERVAL 7 DAY)`	`2026-10-24`

Función	Qué	Ejemplo	Da
`LAST_DAY(f)`	Último día del mes	`LAST_DAY('2026-09-19')`	`2026-09-30`
`DATE_FORMAT(f, formato)`	La fecha como texto, con el formato que se quiera	`DATE_FORMAT(fecha, '%d/%m/%Y')`	`19/09/2026`

Los nombres de días y meses salen en inglés. En español, antes de la consulta: `SET lc_time_names = 'es_ES';` y `DAYNAME` da `sábado`. Dura lo que dure la conexión.

La edad no es `YEAR(CURDATE()) - YEAR(fecha_nacimiento)`: eso suma un año a quien todavía no ha cumplido. Es `TIMESTAMPDIFF(YEAR, fecha_nacimiento, CURDATE())`.

3.4 Control de flujo y nulos

Para que una columna valga una cosa u otra según una condición.

```
-- CASE con condiciones: el más general
SELECT id_partido,
       CASE
         WHEN goles_local > goles_visitante THEN 'local'
         WHEN goles_local < goles_visitante THEN 'visitante'
         ELSE 'empate'
       END AS gana
FROM partido;                    -- 5 local, 6 visitante, 4 empate

-- CASE con valores: compara una expresión con varios valores
SELECT nombre,
       CASE posicion WHEN 'portero' THEN 'POR' WHEN 'cierre' THEN 'CIE' ELSE 'otro' END
FROM jugador;

-- IF(condición, si verdadero, si falso): un CASE de dos ramas, propio de MariaDB y MySQL
SELECT id_gol, IF(minuto <= 20, '1.ª parte', '2.ª parte') AS parte FROM gol;
```

Función	Qué	Ejemplo	Da
`IFNULL(x, otro)`	`x`, o `otro` si `x` es nulo	`IFNULL(pabellon, 'sin pabellón')`	El pabellón, o el texto
`COALESCE(a, b, c, …)`	El primero que no sea nulo. Es el estándar	`COALESCE(NULL, NULL, 'c')`	`c`
`NULLIF(a, b)`	`NULL` si son iguales; si no, `a`	`NULLIF(3, 3)`	`NULL`

`IFNULL` importa en las cuentas: si una columna puede ser nula, `salario + comision` da `NULL` en cuanto falta la comisión. `salario + IFNULL(comision, 0)` no.

4 · Agrupar: `GROUP BY` y `HAVING`

IOC, unidad 4, apartado 2.4 (págs. 267–270).

4.1 Funciones de grupo

Resumen muchas filas en un solo valor.

Función	Qué	Con nulos
`COUNT(*)`	Cuántas filas	Las cuenta todas
`COUNT(columna)`	Cuántos valores no nulos	Se salta los nulos
`COUNT(DISTINCT columna)`	Cuántos valores distintos	Se salta los nulos
`SUM(x)`, `AVG(x)`	Suma, media	Se salta los nulos: la media es de los que tienen valor
`MIN(x)`, `MAX(x)`	Mínimo, máximo. También con texto y fechas	Se salta los nulos
`STDDEV(x)`, `VARIANCE(x)`	Desviación típica, varianza	Se salta los nulos

Sin `GROUP BY`, toda la tabla es un solo grupo y sale una fila:

```
SELECT COUNT(*) AS partidos,
       SUM(goles_local) AS goles_casa,
       SUM(goles_visitante) AS goles_fuera,
       AVG(goles_local + goles_visitante) AS media,
       MAX(goles_local + goles_visitante) AS maximo
FROM partido;
-- 15 · 34 · 34 · 4.5333 · 9

SELECT COUNT(DISTINCT id_jugador) FROM gol;   -- 31 jugadores distintos han marcado
```

4.2 `GROUP BY`

`GROUP BY` hace un grupo por cada valor distinto de una o varias columnas, y las funciones de grupo se calculan **dentro de cada grupo**.

```
SELECT posicion, COUNT(*) AS jugadores
FROM jugador
GROUP BY posicion;
-- ala 18 · cierre 12 · pívot 6 · portero 12

SELECT jornada, SUM(goles_local + goles_visitante) AS goles
FROM partido
GROUP BY jornada;
-- 11 · 20 · 12 · 10 · 15

SELECT id_equipo, posicion, COUNT(*)
FROM jugador
WHERE id_equipo = 1
GROUP BY id_equipo, posicion;   -- un grupo por cada pareja equipo-posición
```

La regla: en el `SELECT` de una consulta agrupada solo puede haber columnas del `GROUP BY` y funciones de grupo. Lo que no está agrupado tiene varios valores en cada grupo: ¿cuál sacar?

```
SELECT posicion, nombre, COUNT(*) FROM jugador GROUP BY posicion;
```

PostgreSQL, Oracle y SQL Server dan error. **MariaDB no**: devuelve un nombre cualquiera de cada grupo, sin avisar. Es así porque la opción `ONLY_FULL_GROUP_BY` no viene activada. El resultado parece bueno y no significa nada.

4.3 `HAVING`

`HAVING` filtra **grupos**, como `WHERE` filtra filas. Va después del `GROUP BY`.

```
SELECT id_jugador, COUNT(*) AS goles
FROM gol
GROUP BY id_jugador
HAVING COUNT(*) >= 4
ORDER BY goles DESC;
-- 7 jugadores: cuatro con 5 goles y tres con 4
```

	`WHERE`	`HAVING`
Filtra	Filas, **antes** de agrupar	Grupos, **después** de agrupar
Puede usar funciones de grupo	No: `WHERE COUNT(*) > 3` da error	Sí
Ejemplo	Solo los goles de la segunda parte	Solo los jugadores con más de 3

Se pueden usar los dos a la vez:

```
-- Partidos con 3 goles o más en la segunda parte
SELECT id_partido, COUNT(*) AS goles
FROM gol
WHERE minuto > 20              -- filas: goles de la segunda parte
GROUP BY id_partido
HAVING COUNT(*) >= 3;          -- grupos: partidos con 3 o más. Salen 7
```

4.4 La sentencia completa

```
SELECT [DISTINCT] columnas, expresiones, funciones de grupo
FROM tablas
[WHERE condición sobre filas]
[GROUP BY columnas]
[HAVING condición sobre grupos]
[ORDER BY columnas [ASC|DESC]]
[LIMIT n [OFFSET m]];
```

Las cláusulas van **siempre en este orden**. Se ejecutan en el del apartado 2.5.

5 · Unión, intersección y diferencia

IOC, unidad 4, apartado 2.5 (págs. 270–272).

Operaciones de conjuntos entre los resultados de dos `SELECT`. Los dos tienen que dar **el mismo número de columnas**, con tipos compatibles una a una. Los títulos salen de la primera consulta.

Operador	Qué da	En la liga
`UNION`	Las filas de las dos consultas, sin repetidos	Equipos que han jugado, de local o de visitante
`UNION ALL`	Las filas de las dos consultas, con repetidos. Más rápido	Cada partido visto desde los dos equipos
`INTERSECT`	Las filas que están en las dos	Jugadores que marcaron en la jornada 1 **y** en la 5
`EXCEPT`	Las filas de la primera que no están en la segunda (Oracle lo llama `MINUS`)	Jugadores que no han marcado

```
SELECT id_local AS id_equipo FROM partido
UNION
SELECT id_visitante FROM partido;            -- 6 filas (con UNION ALL, 30)

SELECT id_jugador FROM jugador
EXCEPT
SELECT id_jugador FROM gol;                  -- 17 jugadores sin gol

SELECT g.id_jugador FROM gol g JOIN partido p ON g.id_partido = p.id_partido WHERE p.jornada = 1
INTERSECT
SELECT g.id_jugador FROM gol g JOIN partido p ON g.id_partido = p.id_partido WHERE p.jornada = 5;
-- 4: Izan Castro, Jorge Lafuente, Raúl Bielsa, Víctor Gil
```

El `ORDER BY` va una sola vez, al final, y ordena el resultado entero.

⚠️ El IOC dice que MySQL no tiene `INTERSECT` ni `EXCEPT`. Era así en 2011. MariaDB los tiene desde la versión 10.3 (2018) y MySQL desde la 8.0.31 (2022).

6 · Combinaciones: `JOIN`

IOC, unidad 4, apartado 2.6 (págs. 272–280).

Las respuestas casi nunca están en una sola tabla. «¿Cómo se llama el máximo goleador?»: los goles están en `gol`, que solo tiene el número del jugador; el nombre está en `jugador`. Hay que **combinar** las dos tablas por la clave ajena.

6.1 El producto cartesiano

Con dos tablas en el `FROM` y nada más, el gestor junta **cada fila de una con cada fila de la otra**:

```
SELECT COUNT(*) FROM jugador, equipo;   -- 288 = 48 jugadores × 6 equipos
```

Cada jugador sale seis veces, una con cada equipo, y solo una de ellas es la suya. Casi nunca es lo que se quiere: es la materia prima de la que se sacan las combinaciones.

6.2 La combinación interna: `JOIN … ON`

Del producto cartesiano, nos quedamos con las filas en las que la clave ajena coincide con la clave primaria:

```
SELECT j.nombre, j.posicion, e.nombre AS equipo
FROM jugador j
JOIN equipo e ON j.id_equipo = e.id_equipo;    -- 48 filas: cada jugador con su equipo
```

Pieza	Qué
`JOIN`	Es lo mismo que `INNER JOIN`, combinación interna. Solo salen las filas con pareja en las dos tablas
`ON`	La condición de combinación: casi siempre `FK = PK`
Alias de tabla	`j` y `e`. Obligatorios en la práctica: las dos tablas tienen una columna `nombre` y hay que decir de cuál

Se pueden encadenar tantos `JOIN` como haga falta. Los máximos goleadores, por nombre:

```
SELECT j.nombre, e.nombre AS equipo, COUNT(*) AS goles
FROM gol g
JOIN jugador j ON g.id_jugador = j.id_jugador
JOIN equipo e  ON j.id_equipo  = e.id_equipo
GROUP BY j.id_jugador, j.nombre, e.nombre
HAVING COUNT(*) >= 4
ORDER BY goles DESC, j.nombre;
-- Gonzalo Lafuente, Izan Castro, Pau Domínguez, Víctor Gil (5);
-- Álvaro Martínez, Pablo Moreno, Unai Campos (4)
```

Se agrupa por `j.id_jugador` y no solo por el nombre: dos jugadores pueden llamarse igual.

Los `JOIN` se encadenan **siguiendo las flechas** del diagrama: `gol → jugador → equipo`. Si se salta una tabla intermedia, no hay columna por la que unir.

6.3 La forma antigua: en el `WHERE`

Antes de SQL-92, la condición de combinación se escribía en el `WHERE`:

```
SELECT j.nombre, e.nombre
FROM jugador j, equipo e
WHERE j.id_equipo = e.id_equipo;     -- el mismo resultado: 48 filas
```

Se sigue viendo en código viejo y funciona en todos los gestores. Pero mezcla la combinación con los filtros, y si se olvida la condición, sale el producto cartesiano sin ningún error. Se usa `JOIN … ON`.

6.4 La misma tabla dos veces

`partido` tiene dos claves ajenas a `equipo`: local y visitante (regla 6 de la teoría 2). Para sacar los dos nombres, `equipo` entra **dos veces**, con dos alias:

```
SELECT p.jornada, l.nombre AS local, p.goles_local, p.goles_visitante, v.nombre AS visitante
FROM partido p
JOIN equipo l ON p.id_local     = l.id_equipo
JOIN equipo v ON p.id_visitante = v.id_equipo
WHERE p.jornada = 1;
```

`local`			`visitante`
CD Moncayo	4	1	Cinco Villas FS
Delicias FS	1	1	Atlético Ebro
Huesca Sala	2	2	Teruel Mudéjar FS

Pasa lo mismo con las relaciones reflexivas: un empleado y su jefe, en la misma tabla. Se llama **autocombinación** (*self join*).

6.5 Combinaciones externas: `LEFT JOIN` y `RIGHT JOIN`

El `JOIN` interno deja fuera las filas sin pareja. A veces son justo las que interesan.

```
-- ¿Cuántos partidos ha jugado en casa cada equipo?
SELECT e.nombre, COUNT(p.id_partido) AS en_casa
FROM equipo e
LEFT JOIN partido p ON p.id_local = e.id_equipo
GROUP BY e.id_equipo, e.nombre;
-- Atlético Ebro sale con 0. Con JOIN normal no saldría: no ha jugado ningún partido en casa
```

Tipo	Qué filas salen
`[INNER] JOIN`	Solo las que tienen pareja en las dos tablas
`LEFT [OUTER] JOIN`	**Todas** las de la tabla de la izquierda; las que no tienen pareja, con `NULL` en las columnas de la derecha
`RIGHT [OUTER] JOIN`	Todas las de la derecha. `A RIGHT JOIN B` es lo mismo que `B LEFT JOIN A`: casi siempre se escribe con `LEFT`
`FULL [OUTER] JOIN`	Todas las de las dos. **MariaDB y MySQL no lo tienen**

Dos detalles del ejemplo:

- `COUNT(p.id_partido)` y no `COUNT(*)`: `COUNT(*)` contaría la fila de Atlético Ebro rellena de nulos como un partido.
- La tabla de la que se quieren **todas** las filas va a la izquierda del `LEFT JOIN`.

Los que no tienen. `LEFT JOIN` más `IS NULL` encuentra las filas sin pareja:

```
-- Jugadores de Teruel que no han marcado
SELECT j.nombre
FROM jugador j
LEFT JOIN gol g ON g.id_jugador = j.id_jugador
WHERE g.id_gol IS NULL AND j.id_equipo = 4;    -- Omar Torres, Martín Royo, Sergio Royo
```

`FULL JOIN` no da error, y debería. ⚠️ En MariaDB, `SELECT … FROM equipo FULL JOIN jugador` se ejecuta: toma `full` como alias de `equipo` y hace un `JOIN` sin condición, el producto cartesiano (288 filas). Para tener todas las filas de las dos tablas, `UNION` de un `LEFT JOIN` y un `RIGHT JOIN`.

6.6 Atajos: `USING` y `NATURAL JOIN`

Cuando las dos columnas se llaman igual:

```
SELECT j.nombre, id_equipo FROM jugador j JOIN equipo USING (id_equipo);   -- 48 filas
```

`USING (columna)` equivale a `ON a.columna = b.columna`, y la columna sale una sola vez.

`NATURAL JOIN` va más allá: une por **todas** las columnas que se llamen igual, sin decir cuáles. Es peligroso:

```
SELECT COUNT(*) FROM jugador NATURAL JOIN equipo;   -- 0 filas
```

`jugador` y `equipo` tienen dos columnas en común, `id_equipo` y `nombre`, y el `NATURAL JOIN` exige que coincidan las dos: que el jugador se llame como su equipo. Mejor no usarlo.

6.7 Condiciones que no son de igualdad

La condición del `ON` puede ser cualquier comparación, y llevar más de una:

```
-- Parejas de compañeros de equipo, sin repetir (A–B y no también B–A)
SELECT COUNT(*)
FROM jugador j1
JOIN jugador j2 ON j1.id_equipo = j2.id_equipo AND j1.dorsal < j2.dorsal;   -- 168 = 6 equipos × 28
```

7 · Subconsultas

IOC, unidad 4, apartado 2.7 (págs. 280–285).

Una **subconsulta** es un `SELECT` dentro de otro, entre paréntesis y sin punto y coma. Se ejecuta primero y su resultado lo usa la consulta de fuera.

7.1 Subconsulta de un valor

Si da una sola fila y una sola columna, se usa como un valor cualquiera, con `=`, `<`, `>`…

```
-- El partido con más goles
SELECT id_partido, jornada, goles_local, goles_visitante
FROM partido
WHERE goles_local + goles_visitante = (SELECT MAX(goles_local + goles_visitante) FROM partido);
-- el 6: 5–4

-- Los pívots del Huesca, sin saber su número
SELECT nombre FROM jugador
WHERE posicion = 'pívot'
  AND id_equipo = (SELECT id_equipo FROM equipo WHERE nombre = 'Huesca Sala');   -- Unai Campos
```

Si la subconsulta puede dar **más de una fila**, el `=` falla:

```
SELECT nombre FROM jugador
WHERE id_equipo = (SELECT id_equipo FROM equipo WHERE ciudad = 'Zaragoza');
-- ERROR 1242: Subquery returns more than 1 row (Zaragoza tiene dos equipos)
```

Da igual que hoy dé una: si **puede** dar varias, se usa `IN`.

7.2 `IN` y `NOT IN`

Con una subconsulta que da una columna y varias filas:

```
SELECT COUNT(*) FROM jugador WHERE id_jugador IN (SELECT id_jugador FROM gol);       -- 31
SELECT COUNT(*) FROM jugador WHERE id_jugador NOT IN (SELECT id_jugador FROM gol);   -- 17
```

`NOT IN` y los nulos. Si la lista tiene un `NULL`, `NOT IN` no devuelve **nada**: `WHERE id_equipo NOT IN (1, NULL)` da 0 filas. «No es 1 ni es desconocido» nunca es verdadero del todo. Si la subconsulta puede dar nulos, mejor `NOT EXISTS`.

Se pueden comparar varias columnas a la vez, entre paréntesis y en el mismo orden:

```
SELECT nombre FROM jugador
WHERE (id_equipo, dorsal) IN (SELECT 4, 1 UNION SELECT 5, 9);   -- Omar Torres, Lucas Zapater
```

7.3 `ANY` y `ALL`

Comparan un valor con **todos** los de la lista (`ALL`) o con **alguno** (`ANY`):

```
-- Partidos con más goles que cualquiera de la jornada 1 (que tuvo 5, 2 y 4)
SELECT id_partido FROM partido
WHERE goles_local + goles_visitante > ALL (SELECT goles_local + goles_visitante
                                           FROM partido WHERE jornada = 1);
-- 5, 6, 12, 15: los que tienen más de 5
```

Expresión	Equivale a
`> ALL (…)`	`> (SELECT MAX(…))`
`> ANY (…)`	`> (SELECT MIN(…))`
`= ANY (…)`	`IN (…)`
`<> ALL (…)`	`NOT IN (…)`

⚠️ El IOC pone que `=ANY` equivale a `NOT IN`: es una errata. Es `<> ALL`.

Se usan poco: con `MAX`, `MIN` e `IN` se lee mejor.

7.4 Subconsultas correlacionadas

La subconsulta usa una columna de la consulta de fuera, así que se repite **para cada fila**:

```
-- El jugador más veterano de cada equipo
SELECT e.nombre AS equipo, j.nombre, j.fecha_nacimiento
FROM jugador j
JOIN equipo e ON e.id_equipo = j.id_equipo
WHERE j.fecha_nacimiento = (SELECT MIN(j2.fecha_nacimiento)
                           FROM jugador j2
                           WHERE j2.id_equipo = j.id_equipo);   -- j es la de fuera
-- 6 filas, una por equipo: Manuel Herrero, Hamza Serrano, Andrei Serrano, Daniel Gil,
-- Youssef Sánchez, Marco El Amrani
```

Para cada jugador `j`, la subconsulta busca la fecha mínima **de su equipo**. Los alias `j` y `j2` distinguen la tabla de fuera de la de dentro: sin ellos, `id_equipo = id_equipo` sería siempre cierto.

7.5 `EXISTS` y `NOT EXISTS`

`EXISTS (subconsulta)` es verdadero si la subconsulta da alguna fila. Casi siempre es correlacionada. No importa qué columnas devuelva: se escribe `SELECT 1`.

```
-- Equipos que no han jugado en casa
SELECT nombre FROM equipo e
WHERE NOT EXISTS (SELECT 1 FROM partido p WHERE p.id_local = e.id_equipo);   -- Atlético Ebro

-- Jugadores sin goles: el mismo resultado que NOT IN, sin el problema de los nulos
SELECT COUNT(*) FROM jugador j
WHERE NOT EXISTS (SELECT 1 FROM gol g WHERE g.id_jugador = j.id_jugador);    -- 17
```

7.6 Subconsultas en el `FROM` y en el `SELECT`

En el `FROM`, el resultado de una consulta se usa como si fuera una tabla. Necesita un alias. Sirve para agrupar dos veces, o para preparar los datos antes. La clasificación de la liga:

```
SELECT e.nombre,
       SUM(CASE WHEN r.gf > r.gc THEN 3 WHEN r.gf = r.gc THEN 1 ELSE 0 END) AS puntos,
       SUM(r.gf) - SUM(r.gc) AS diferencia
FROM (
  SELECT id_local AS id_equipo, goles_local AS gf, goles_visitante AS gc FROM partido
  UNION ALL
  SELECT id_visitante, goles_visitante, goles_local FROM partido
) r                                      -- cada partido, una vez desde cada equipo
JOIN equipo e ON r.id_equipo = e.id_equipo
GROUP BY e.id_equipo, e.nombre
ORDER BY puntos DESC, diferencia DESC;
```

nombre	puntos	diferencia
Teruel Mudéjar FS	10	6
Delicias FS	10	2
CD Moncayo	7	2
Atlético Ebro	6	0
Huesca Sala	4	−4

nombre	puntos	diferencia
Cinco Villas FS	4	−6

La subconsulta `r` le da la vuelta a `partido`: en vez de local y visitante, «este equipo, sus goles y los del rival». Con eso, los puntos son una suma.

En el `SELECT`, una subconsulta de un valor calcula una columna:

```
SELECT nombre,
       (SELECT COUNT(*) FROM jugador j WHERE j.id_equipo = e.id_equipo) AS jugadores
FROM equipo e;
```

7.7 ¿Subconsulta o `JOIN`?

Muchas preguntas se contestan de las dos maneras.

Si…	Mejor
Hay que **mostrar** columnas de las dos tablas	`JOIN`
La otra tabla solo sirve para **filtrar**	Subconsulta con `IN` o `EXISTS`
Se compara con un total o una media	Subconsulta de un valor
«Los que no tienen»	`NOT EXISTS` o `LEFT JOIN … IS NULL`

Lo que ha cambiado desde el IOC

El IOC es de 2011 y usa MySQL 5. Lo que ya no es así, o no es así en MariaDB:

Dice el IOC	Hoy, en MariaDB
Restar dos fechas con `-` da los días	Da un número sin sentido (98 en lugar de 28). Se usa `DATEDIFF` o `TIMESTAMPDIFF`
`NULL` va al final al ordenar	Va al principio en `ASC`
MySQL no tiene `INTERSECT` ni `EXCEPT`	MariaDB los tiene desde 2018, MySQL desde 2022
`=ANY` equivale a `NOT IN`	Errata: `= ANY` es `IN`; `<> ALL` es `NOT IN`
Se trabaja con MySQL Workbench	En clase, MariaDB y DBeaver
`ZEROFILL` y `INT(4)` para el ancho de visualización	Desaconsejados: en MySQL 8 ya están obsoletos

Anexo · MariaDB y DBeaver

Sustituye al anexo del IOC, que instala MySQL Workbench (págs. 285–302).

Paso	Cómo
Conectar	DBeaver → *Nueva conexión* → MariaDB → servidor `localhost`, puerto `3306`, usuario `root` y su contraseña

Paso	Cómo
Crear la base	Botón derecho sobre la conexión → *Crear* → *Nueva base de datos*, con juego de caracteres `utf8mb4`. O en SQL: `CREATE DATABASE liga CHARACTER SET utf8mb4;`
Abrir un editor	Seleccionar la base → *SQL Editor* → *New SQL script*. Arriba, en la barra del editor, tiene que poner la base buena
Cargar `liga.sql`	Abrir el fichero en un editor y ejecutarlo como script: **Alt+X**. Borra y crea la base de cero cada vez
Ejecutar una consulta	Cursor dentro de la consulta → **Ctrl+Intro**. Solo esa
Ejecutar todo el fichero	**Alt+X**
Ver las tablas	**F5** sobre la base, para refrescar
Ver la estructura	`DESCRIBE tabla;`, o doble clic en la tabla → pestaña *Columnas*

Errores frecuentes:

Mensaje	Qué pasa
`No database selected` o `Unknown table`	El editor no está sobre la base `liga`. Se elige arriba, o se ejecuta `USE liga;`
`You have an error in your SQL syntax … near '…'`	Falta o sobra algo justo **antes** del trozo que cita: una coma, un paréntesis, una comilla
`Unknown column 'x' in 'WHERE'`	Columna mal escrita, o un alias usado en el `WHERE`
`Column 'nombre' in SELECT is ambiguous`	Dos tablas del `JOIN` tienen esa columna: falta el alias (`j.nombre`)
`Subquery returns more than 1 row`	Un `=` con una subconsulta que da varias filas: se cambia por `IN`
Salen dos consultas como una	Falta el `;` entre ellas
Salen 288 filas donde se esperaban 48	Un `JOIN` sin `ON`, o una coma en el `FROM` sin condición en el `WHERE`

Glosario

Término	Qué es
SQL	*Structured Query Language*. El lenguaje estándar de las bases de datos relacionales
DDL, DML, DQL, DCL, TCL	Los grupos de sentencias SQL: definición, manipulación, consulta, control y transacciones
Cláusula	Cada parte de una sentencia: `SELECT`, `FROM`, `WHERE`…
Alias	Nombre provisional para una columna o una tabla dentro de una consulta
Expresión	Cuenta con columnas, valores y funciones que da un valor por fila
Colación	Reglas de orden y comparación del texto de un idioma: si las tildes y mayúsculas cuentan, dónde va la ñ
`NULL`	Valor desconocido o que no existe. No es cero ni texto vacío. Se pregunta con `IS NULL`
Función de grupo	Resume varias filas en un valor: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`

Término	Qué es
Producto cartesiano	Todas las combinaciones de las filas de dos tablas
Combinación interna (`INNER JOIN`)	Las filas que tienen pareja en las dos tablas según una condición
Combinación externa (`LEFT`/`RIGHT JOIN`)	Además, las filas sin pareja de una de las tablas, con nulos en la otra
Autocombinación	Una tabla combinada consigo misma, con dos alias
Subconsulta	`SELECT` dentro de otra sentencia, entre paréntesis
Subconsulta correlacionada	La que usa columnas de la consulta de fuera y se repite para cada fila
Tabla derivada	Subconsulta en el `FROM`, usada como una tabla

Autoevaluación

Sobre la base `liga`. Primero en papel; luego, comprobadlo en DBeaver.

1. ¿A qué grupo de sentencias pertenece cada una: `CREATE TABLE`, `UPDATE`, `GRANT`, `SELECT`, `ROLLBACK`?
2. ¿Qué tipo usaríais para el precio de una entrada, en euros con céntimos? ¿Por qué no `FLOAT`?
3. ¿Qué devuelve `SELECT nombre FROM jugador ORDER BY fecha_nacimiento DESC LIMIT 1;`?
4. Esta consulta da error. ¿Por qué, y cómo se arregla? `SELECT id_partido, goles_local - goles_visitante AS dif FROM partido WHERE dif > 2;`
5. ¿Cuántas filas da `SELECT * FROM equipo WHERE pabellon = NULL;`, haya o no equipos sin pabellón?
6. Escribid la consulta: jugadores nacidos en el año 2000 o después cuyo apellido termine en «ez».
7. ¿Por qué `COUNT(*)` y `COUNT(pabellon)` pueden dar números distintos sobre la misma tabla?
8. ¿Qué diferencia hay entre `WHERE` y `HAVING`? Poned un ejemplo de la liga que use los dos.
9. ¿Qué devuelve `SELECT posicion, nombre, COUNT(*) FROM jugador GROUP BY posicion;` en MariaDB? ¿Y en PostgreSQL?
10. ¿Cuántas filas da `SELECT * FROM jugador, partido;`? ¿Para qué sirve eso?
11. Escribid la consulta: los goles del partido 15, con el minuto y el nombre de quien lo marcó.
12. ¿Por qué hace falta escribir `equipo` dos veces para sacar los resultados con los nombres de los dos equipos?
13. Con `JOIN`, ¿sale Atlético Ebro al contar los partidos que cada equipo ha jugado en casa? ¿Y con `LEFT JOIN`? ¿Por qué hay que contar `COUNT(p.id_partido)` y no `COUNT(*)`?
14. ¿Por qué `jugador NATURAL JOIN equipo` da 0 filas?
15. Esta consulta da error. ¿Por qué, y cómo se arregla? `SELECT nombre FROM equipo WHERE id_equipo = (SELECT id_local FROM partido WHERE jornada = 1);`
16. Escribid, con una subconsulta, los jugadores que han marcado en el minuto 40.
17. ¿Qué diferencia hay entre `UNION` y `UNION ALL`? ¿Cuál hace falta en la clasificación, y por qué?

Respuestas

1. `CREATE TABLE`, DDL. `UPDATE`, DML. `GRANT`, DCL. `SELECT`, DQL (o DML, según el libro). `ROLLBACK`, TCL.
2. `DECIMAL(6,2)`, por ejemplo. `FLOAT` es aproximado: `0.1 + 0.2` no da exactamente `0.3`, y con dinero no se admiten redondeos que nadie ha pedido.
3. El jugador **más joven**: la fecha de nacimiento más alta es la más reciente. Javier Herrero (2005-09-04).
4. El `WHERE` se ejecuta antes que el `SELECT` y el alias `dif` todavía no existe. Se repite la expresión: `WHERE goles_local - goles_visitante > 2`.
5. Ninguna, siempre. `pabellon = NULL` da `NULL`, nunca verdadero. Se escribe `pabellon IS NULL`.
6. `SELECT nombre, fecha_nacimiento FROM jugador WHERE fecha_nacimiento >= '2000-01-01' AND nombre LIKE '%ez';`
7. `COUNT(*)` cuenta filas; `COUNT(pabellon)`, solo las que tienen pabellón. Si algún equipo lo tiene a `NULL`, la segunda es menor.
8. `WHERE` filtra filas antes de agrupar; `HAVING`, grupos después. Partidos con 3 o más goles en la segunda parte: `SELECT id_partido, COUNT(*) FROM gol WHERE minuto > 20 GROUP BY id_partido HAVING COUNT(*) >= 3;`
9. En MariaDB, una fila por posición con un nombre cualquiera del grupo, sin avisar. En PostgreSQL, un error: `nombre` no está en el `GROUP BY` ni dentro de una función de grupo.
10. 48 × 15 = 720. Es el producto cartesiano: cada jugador con cada partido. Por sí solo no sirve; las combinaciones salen de filtrarlo.
11. `SELECT g.minuto, j.nombre FROM gol g JOIN jugador j ON g.id_jugador = j.id_jugador WHERE g.id_partido = 15 ORDER BY g.minuto;` (8 filas).
12. Porque `partido` tiene dos claves ajenas a `equipo`, local y visitante, y cada una necesita su propia combinación. Con una sola copia de `equipo`, el mismo equipo tendría que ser local y visitante a la vez.
13. Con `JOIN`, no: no tiene ningún partido en casa y se queda sin pareja. Con `LEFT JOIN`, sí, con un 0. `COUNT(*)` contaría su fila rellena de nulos y diría 1; `COUNT(p.id_partido)` se salta los nulos.
14. Porque une por todas las columnas que se llaman igual, y `jugador` y `equipo` comparten `id_equipo` y `nombre`: solo saldría un jugador que se llamara como su equipo.
15. La subconsulta da tres filas (tres partidos en la jornada 1) y el `=` solo admite una. Se cambia por `IN`.
16. `SELECT nombre FROM jugador WHERE id_jugador IN (SELECT id_jugador FROM gol WHERE minuto = 40);` (Hugo Bielsa y Pau Domínguez).
17. `UNION` quita las filas repetidas; `UNION ALL`, no. En la clasificación, `UNION ALL`: si un equipo gana dos partidos 2–1, las dos filas son iguales y `UNION` dejaría una sola, y se perderían puntos.